



IBM BOB 2.0 HACKATHON · 27/09/2026

# NodeFlow

El caso real de un workflow de desarrollo asistido por **IBM Bob 2.0**: transformar una idea en un **contrato versionado**, implementarlo con Bob y revisar la entrega mediante pruebas y auditoría independiente.

Tauri · Rust · React Flow · TypeScript · repo público · demo web sin instalar

# Un pedido ambiguo no produce una entrega verificable — y el retrabajo se paga tres veces

## Lo que cuesta hoy

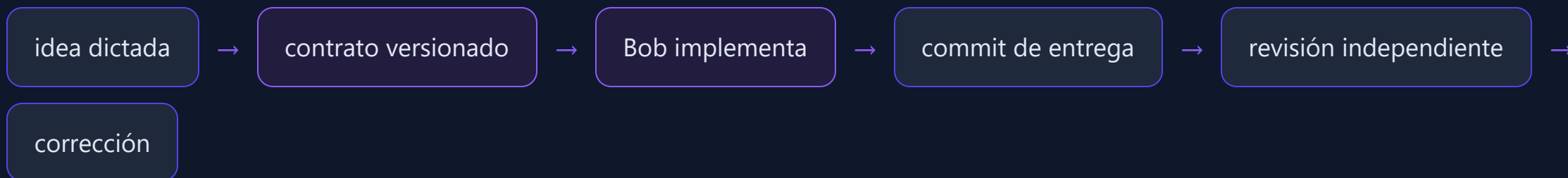
- Al **implementar**: se construye de más o de menos, porque el alcance no está escrito.
- Al **revisar**: no hay contra qué comparar; la revisión se vuelve opinión.
- Al **corregir**: el defecto aparece tarde, cuando ya se apoyó código encima.

## Lo que hicimos distinto

La idea se escribe como **contrato commiteado antes de tocar código**. El agente implementa contra ese contrato y una **revisión independiente** audita la entrega contra el mismo texto.

Cuando el contrato no fijaba algo —la cardinalidad de un evento— la auditoría lo encontró **antes** de que llegara a producción.

## De la idea al código, con **evidencia en cada paso**



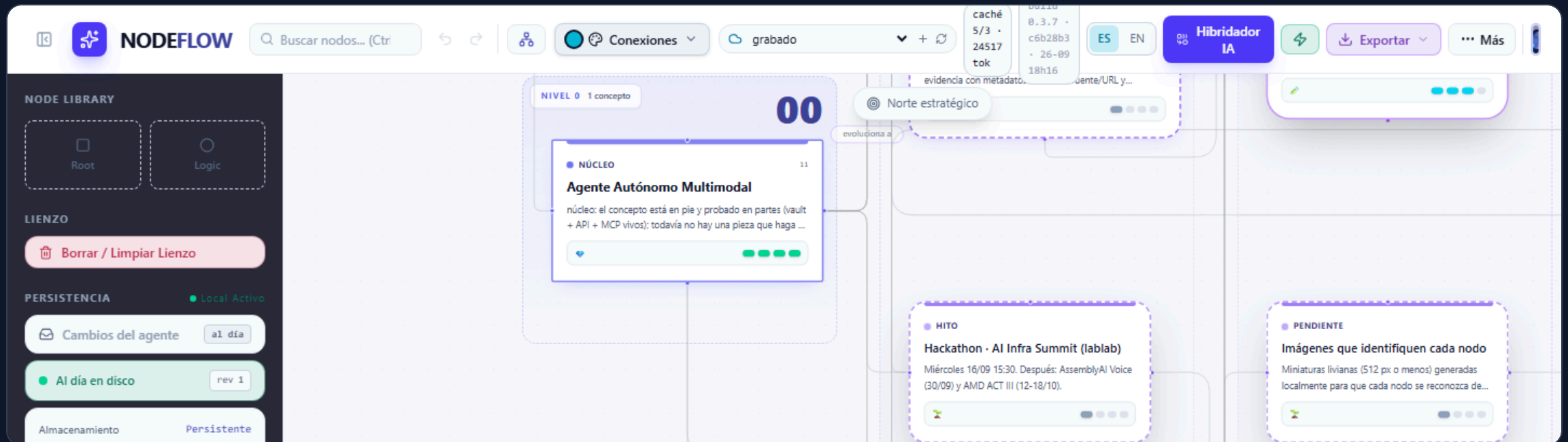
Un pedido completo, tal como está en el historial de git:

- Contrato del segmentador de voz commiteado **antes** de tocar código: 00edcf6
- Bob implementa el módulo y su endpoint: entrega en cff0baa
- La revisión independiente encuentra un **defecto real de temporalidad**: corrección en 0e059ed

Los pasos son **manuales** y deliberados: no es una integración automática de voz a código.

# Un lienzo que se dibuja mientras hablas

App de escritorio (Tauri + Rust + React Flow): un **segmentador determinista** del backend decide si lo que dijiste es idea, corrección o ruido; aparece un **borrador**; al cerrar el turno cada tema se vuelve un **nodo real** —encadenado, movable y conectable— y **Ctrl+Z** revierte. El flujo sin fricción no es aprobar antes: es poder deshacer después.



## Bob escribió el motor de voz. La cadena lo prueba.

### Qué implementó Bob

- El **segmentador de voz** del backend (Rust) y su endpoint.
- El **cliente del parcial** en el frontend.
- Los **arreglos de UX** que salieron de la primera prueba con audio real.

### Dónde está la evidencia

- Los **contratos** commiteados antes del código:  
[docs/hackathon/PEDIDO-0\\*.md](#)
- Las **capturas del resumen de consumo** de cada tarea:  
[bob\\_sessions/](#)
- Qué archivos tocó y qué no:  
[docs/hackathon/EVIDENCIA-BOB.md](#)

Lo que **no** afirmamos: no atribuimos a Bob paralelismo ni subagentes. Sólo lo acreditado —contratos, commits, capturas—, ni una línea más.

## Números que se pueden volver a correr

324/324

tests de Rust

135/135

tests del front

23

PRs mergeados

48<sub>h</sub>

de construcción

### Un caso que medimos

Una **tarea entera** encargada a un agente produjo **0 archivos en 80 llamadas**. La misma cosa, partida como **pieza** (una función pura con firma exacta y casos enumerados), se entregó en **1 intento** con 12 tests en verde.

### Los límites, dichos de frente

- 324/324 y 135/135 son **resultados de las suites**, no un ahorro medido.
- El caso de arriba es **un caso observado**, no un experimento controlado.
- El proceso está aplicado **a este proyecto**: no demuestra productividad general.

## El proceso, reutilizable. El producto, funcionando.

### Probarlo

**Repo** · [github.com/NeonEden/nodeflow-ibm-bob](https://github.com/NeonEden/nodeflow-ibm-bob)

**Demo web** · [nodeflow-ibm-bob.vercel.app](https://nodeflow-ibm-bob.vercel.app) (simula la voz: sin backend Rust no hay micrófono)

**App de escritorio** · release firmado para Windows, o `npx tauri dev`

### Qué llevamos

Un workflow donde la idea queda escrita antes del código, el agente implementa contra ese texto y la entrega se audita de forma independiente — con **IBM Bob 2.0 escribiendo el motor** y la evidencia de cada paso en el repositorio.